A Private Blockchain for Jointly Signed Negotiated Rates Between Providers and Payers

Abhinav Srivastava[1]

[1]**Independent Researcher; abhinavmirdev@gmail.com**

Abstract

Under two US rules, hospitals and health insurers each publish the prices they negotiated, in separate files. In 2026 a study of 450 hospitals and two national insurers found that both sides published the same price for only about 10% of services compared. We present CORAL, a shared ledger on which the provider and the insurer both sign each negotiated price. The signed record holds the price, the insurance plan, the billing code, whether the service is inpatient or outpatient, and its dates. Each published file is checked against the signed records; a price that differs from its record marks that file as wrong. We signed one record for each rate in one provider's file and checked that file against the ledger. We compared five providers' files with one national insurer's file. A synthetic run traces a changed price in a copy of either file to that copy.

Introduction

Since 2021, each hospital in the United States must publish a machine-readable file of its standard charges, including the rate it negotiated with each payer and plan, under the Hospital Price Transparency rule (45 CFR Part 180) [1]. Since 2022, each group health plan and issuer must publish in-network negotiated rates for every provider under the Transparency in Coverage rule [2]. The two programs publish the same contract terms from two sides (Figure 1a). Each file is produced by one party under its own rule, and no clause of either rule asks the other party to review or confirm it [1, 2]. The hospital attests to its own file, under a statement added for CY2024 and extended for CY2026 [3, 4]; the payer's yearly attestation concerns gag clauses in its contracts, not its file [5].

The two sides do not agree. A 2026 industry report compared the files of 450 hospitals with the files of two national payers on 16 common codes and found that 9.0% of inpatient pairs, 10.8% of gastroenterology pairs, and 11.7% of orthopedic pairs matched exactly; about half fell within 10%; and 17.9% of inpatient and gastroenterology pairs differed by more than 50% [6]. The comparison cannot say which file is correct, because neither file links a line to a contract version or to the other party.

A negotiated rate is a contract term that two parties agreed. CORAL (Committed Ratesheet on Ledger) is a registry on which a provider and a payer jointly sign each negotiated rate before either publishes it (Figure 1b), so that a published line can be checked against a record neither party wrote alone. The files cite the record; a verifier with a file and read access to the registry checks each line with one lookup, and a line that departs from the record is attributed to the file that departs.

The paper gives the record format, with a key that both public formats map to, and the registry contract, whose key hierarchy judges validity at commit time, rejects backdated revocations, delays root replacement under a veto, and keeps one live record per key. We sign and commit all 262,535 rates of one provider's public file, verify every published line with a standard code and a value, and measure throughput, bytes, and storage. A second run compares five providers' files with one national payer's file on the 16 codes of the 2026 report and traces synthetic alterations of two copies of the signed values to the copy that carries them. Both signatures in every run come from prototype keys, so the runs show cost and coverage, not that any file is correct.

Background

The two files

The provider file, in the CMS tall CSV format, has one row per item and payer plan; the columns that identify a rate and the columns that give its value are the left column of Figure 2 [7]. It must be refreshed at least once a year. The payer file, in the Transparency in Coverage JSON schema, has one item per billing code, with negotiated rates for groups of providers identified by NPI and tax identification number, each with a negotiated type, a billing class, place-of-service codes, modifiers, and an expiration date [8]. It must be refreshed every month.

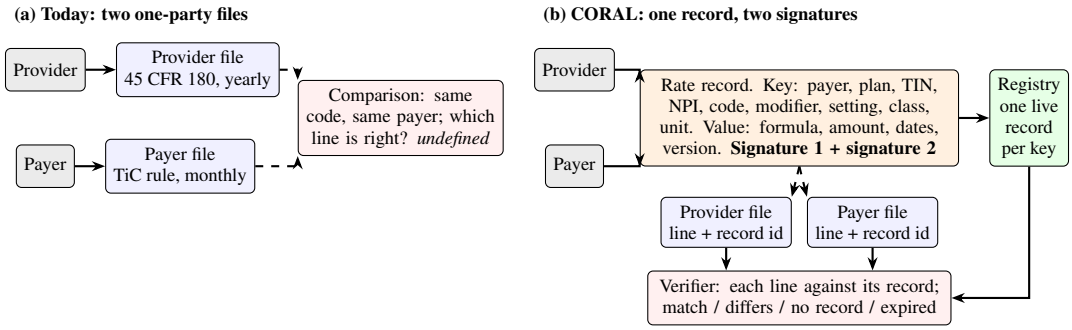


Figure 1: Today each party publishes its own file, and a comparison of the two has no reference to decide which line is right. With CORAL the two parties sign one record per rate; each file cites the record, and a verifier checks each published line against it.

What a negotiated rate is

The chargemaster is the provider's list price for every item, set by the provider alone and never negotiated; the CMS file calls it the gross charge. The negotiated rate is a contract term: the price the provider and the payer agreed for one code under one plan, the payer-specific negotiated charge of the CMS file. The record covers three forms of it: a fixed amount, a percent of the chargemaster price, and a percent of a reference fee schedule. In the provider file we profile, 37.7% of the rows that carry a value are percent-of-charges rates, so their dollar value moves whenever the provider changes the chargemaster, which the payer does not sign. Contracts also carry terms the record does not cover: lesser-of-charges clauses, outlier and stop-loss provisions, carve-outs at a percent of invoice, tiered per diems, bundled and capitated payment, and escalators.

Why the files disagree

The identifying fields do not line up: the provider file has setting where the payer file has place-of-service codes, the provider file has a drug unit and the payer file has none, and plan names are free text on one side and plan clusters on the other. Each file is a snapshot with one date, the refresh cycles differ, and contracts change mid-year. The provider file holds many items under one billing code, with different list prices and sometimes different fixed amounts, and a percent-of-charges line gives a dollar result that depends on the chargemaster version in force when the file was built. Comparing the files resolves none of this, because neither says what the contract term was.

Related work

Price transparency. Sinaiko and Rosenthal, and later Mehrotra and colleagues, set out what price publication can and cannot change [9, 10]. Gondi and colleagues and Jiang and colleagues measured early and national hospital compliance with the rule [11, 12]. Wang and colleagues found Transparency in Coverage prices consistent with other commercial sources [13]. Henderson and Mouslim cross-validated one regional insurer's file with 26 hospitals' files on maternity codes: 16.3% of hospital observations appeared in both files, and where both had a price, 77.4% agreed to the cent and 84.4% fell within 10% [14]. Two advocacy and industry groups publish recurring reports on file availability and quality [15, 16]; the 2026 comparison is the direct motivation for this paper [6]. Every one of these studies works from files that a single party produced.

One-party attestation and two-party estimates. The CY2024 OPPS final rule added an affirmation statement to the hospital file [3]; the CY2026 rule extends it, names a senior official, replaces the estimated allowed amount with percentile allowed amounts from remittance data, and adds the hospital's Type 2 NPIs to the file [4]. Each binds the hospital to its own file only. The No Surprises Act's good faith estimate and advanced explanation of benefits move a price estimate from provider to plan to patient for one visit [17], carried in FHIR by the HL7 Da Vinci Patient Cost Transparency guide [18]; it is a two-party exchange, but per visit, and neither party signs the other's figure.

Ledgers in health care. Kuo, Kim, and Ohno-Machado give the conditions under which a ledger adds value over a database: several parties that do not trust one operator, a shared need for an append-only history, and no acceptable neutral host [19]. Claims and directory networks on permissioned ledgers exist: a national clearinghouse's claims network on Hyperledger Fabric [20], a payer consortium's provider directory [21], a practitioner credentials exchange [22], and a smart-contract store for insurance records [23]. What each holds is a fact that one party asserts.

Transparency logs. Certificate Transparency [24, 25], Trillian [26], and the IETF SCITT architecture [27] give an

append-only log under one operator with inclusion proofs. PeerReview [28] and SUNDR [29] detect equivocation by a log operator. Zieglmeier shows that an append to a chain proves the sender's action and not the counterparty's receipt [30]; CORAL answers this with the counterparty's signature inside the record. A one-operator log would serve CORAL if providers, payers, and the regulator accepted one operator; the Discussion returns to this.

Signing. EIP-712 defines typed structured data signing, which shows the signer the fields and binds the signature to one deployment [31]; every record uses it.

Methods

The record

A rate record has a key and a value (Figure 2). The key is the set of fields that identify one rate; two records equal on all key fields are the same rate. The value gives the formula (fixed, percent of charges, or percent of reference), the amount in cents or basis points, the start and end dates, a contract version label, a reference version (the chargemaster or fee-schedule version a percent rate applies to), and the id of the record this one supersedes, if any.

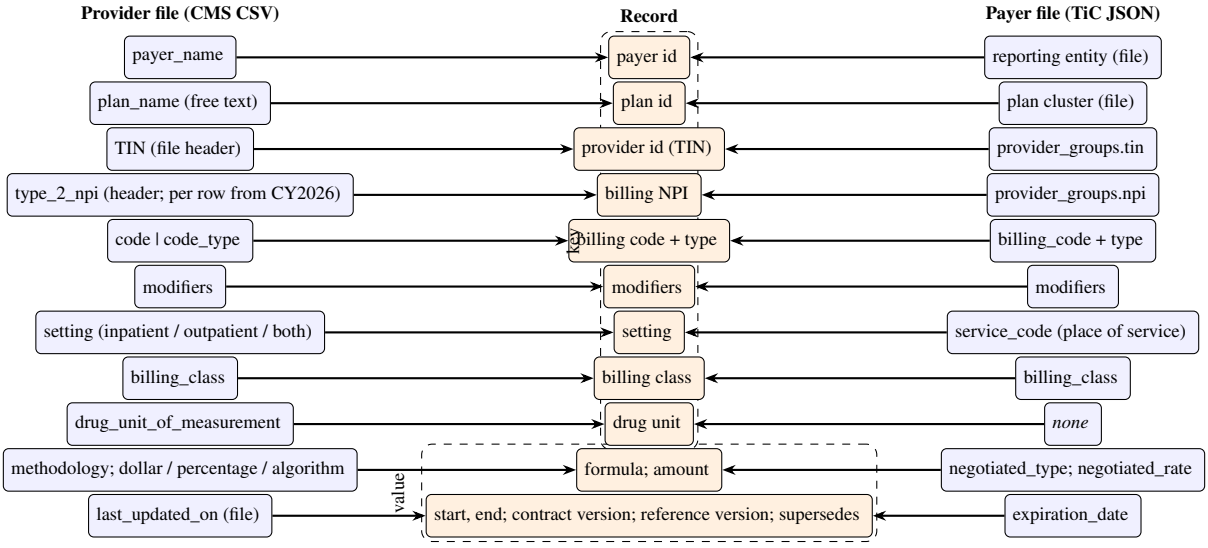


Figure 2: The rate record and where each field comes from in the two public formats. The contract version and the supersedes link exist in neither file; they are what make a verdict possible.

The setting and billing class fields come from the CMS file. The payer file carries billing class directly and, for professional prices, setting through place-of-service codes: 21 is inpatient; 19, 22, 23, and 24 are outpatient. An institutional price carries no place of service: a DRG is inpatient, and any other institutional price is taken as applying in both settings, as the CMS file's "both" does; the parser counts how often it did so, and counts a professional price with no place of service as unknown. The payer file has no drug unit.

Entity, Delegation, Revocation, and Chargemaster Version records support the rate record. An Entity binds an entity id to a root public key. A Delegation authorizes a signing key for an entity over a time window. A Revocation ends a signing key from a time at or after the block that carries it. A Chargemaster Version is the provider's signature over a version label, the SHA-256 of the chargemaster file, and an effective date; a percent-of-charges record names it, so the dollar result recomputes from signed inputs.

Identity and keys

The provider's identity root is its tax identification number, because contracts are signed at that level and the payer file carries it; a system with several TINs holds one root per TIN. The payer's root is an identifier the payer file schema allows, the HIOS issuer id or, for a self-funded plan sponsor, its EIN; the national health plan identifier was rescinded in 2019 [32]. The registrar binds an entity id to a root key once, after out-of-band proofing, and never signs a rate (Figure 3). Neither rule sends a file to a regulator: each party posts its file on its own site, CMS enforces the hospital rule, and HHS, Labor, and Treasury enforce the Transparency in Coverage rule, with states as primary enforcers for

issuers [2]. The registrar needs identity proofing only, which CMS already does for hospitals through PECOS and for issuers through HIOS, so the prototype makes CMS the registrar.

Each entity holds one root key offline and delegates one or more signing keys to the system that produces its rate sheet, which for many providers is a vendor's system; the delegation is on the registry and a revocation ends it. The root signs delegations and revocations only. It cannot sign a rate, because the registry checks a rate signer against delegations and a root has none. A verifier answers "was key K authorized for entity E at time T" from the registry alone.

Validity is judged at the block timestamp of the commit, a time that consensus sets within the block period and the signer cannot choose. A revocation must take effect at or after the block that carries it, so a root cannot backdate a revocation to before a record its signing key committed; records committed before a revocation stand. Root replacement after loss is proposed by the registrar, takes effect after seven days, and the current root can veto it during that window with its signature. The delay gives the entity notice before the registrar can re-key it. Loss is covered and theft is not: a holder of a stolen root can veto its own replacement, and a registrar override after a longer delay is a design we have not built.

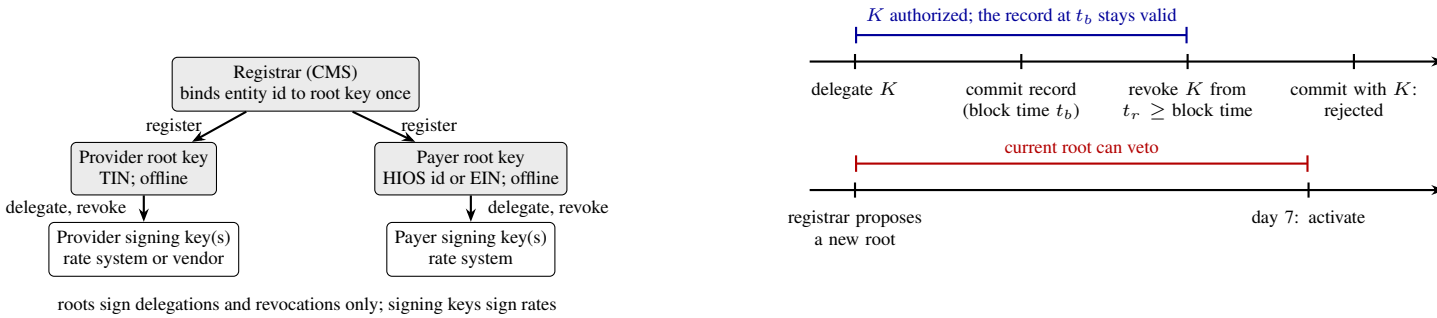


Figure 3: Keys and time rules. Left: the registrar binds each entity to a root; roots delegate signing keys. Right: validity is judged at block time; a revocation cannot start before its block; root replacement waits seven days under a veto.

The registry

The registry is one contract on a permissioned Ethereum-compatible network (Figure 4). Each action carries EIP-712 signatures over a typed structure; the domain binds the chain id and contract address, so a signature for one deployment is not valid on another. Both parties sign the same structure hash, and that hash is the record id. Identifier fields are 32-byte strings; a longer identifier is stored as its hash, and the published file must carry the full text. Anyone can relay the transaction, and the relayer has no authority. Records commit one at a time or in a batch that succeeds or fails as a whole.

One record is live per key at any time. A new record for a key whose window overlaps a live record is rejected, unless it names that record in its supersedes field; the old record then ends where the new one starts. Both parties sign the new record, so an amendment carries both signatures and the history of a key is a chain of records. At any block height a verifier finds one answer for one date; a superseding record may start in the past, so a verdict is reported as of a block height and can change when a later record supersedes.

The prototype network is Hyperledger Besu 26.10.2 with IBFT 2.0 consensus [33], a two-second block period, and four validators, which stand for CMS, a provider association, a payer association, and an auditor. Validators order transactions; they cannot sign a rate. In every run all four validators ran in Docker on one laptop with a 10-core ARM processor and 64 GB of memory, so the throughput figures carry no network latency. Read access in the prototype is open to any node; a deployment sets it as policy.

Verification

A verifier reads the registry once into an index by key, then checks each published line with one lookup; Figure 5 gives the verdicts. For a fixed rate the expected amount is the signed value. For a percent-of-charges rate it is the signed percent of the list price for that code, setting, and unit in the chargemaster version the record names; a provider line that publishes the percent is compared on the percent, and a line that publishes only the dollar result is compared on the recomputed dollar. When both files are verified, a pair of lines resolves to both match the record, payer file departs, provider file departs, both depart, or unsigned. The two files alone give no such verdict.

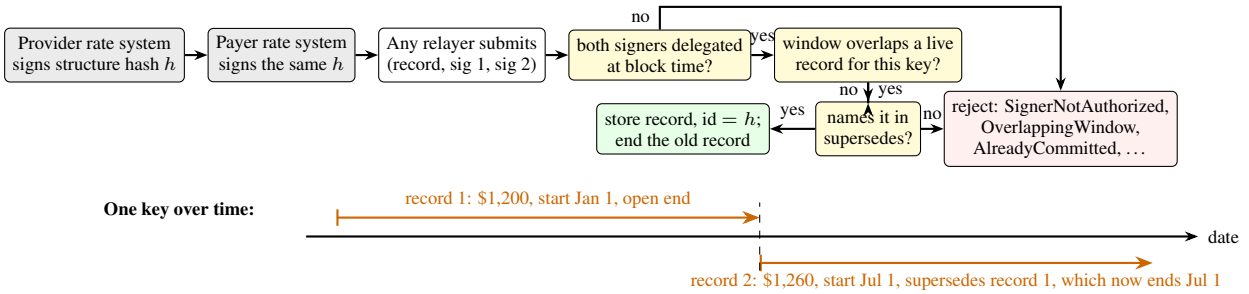


Figure 4: Commit path and supersession. The contract checks both signers against delegations at block time, rejects an overlapping window unless the new record names the live one, and keeps one live record per key.

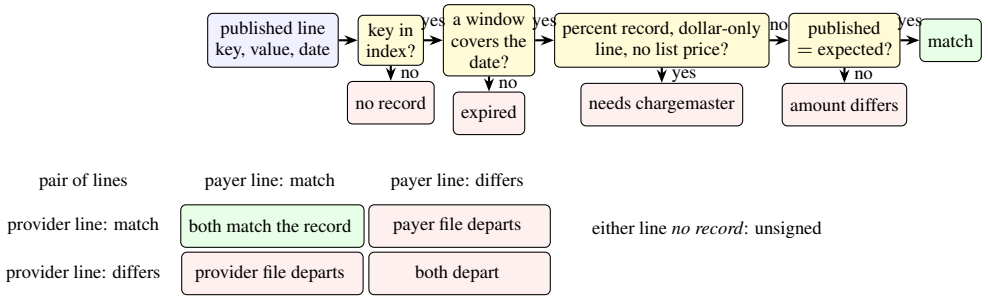


Figure 5: Verdict for one published line (top) and for a pair of lines from the two files (bottom). A difference between the files is attributed to the line that departs from the signed record.

Evaluation design

Full-sheet run. We take the public file of provider A, a hospital in Colorado (file dated 2025-11-01; CMS format 3.0.0). Providers are lettered A to E in this paper; payers are not named. Every payer named in the file is registered with prototype keys. Every row with a percentage becomes a percent-of-charges record that names a signed chargemaster version; every other row with a dollar amount becomes a fixed record; rows with only an algorithm are counted. One record is signed per key, from the first row in file order; a later row with the same key and a different value is a key collision and is counted. We then verify every row with a standard code and a value against the registry, and again after altering 1% of those rows.

Pair run on real data. We take five providers' files (Table 2) and one Transparency in Coverage in-network file of a national commercial payer, payer P (one plan cluster, dated 2026-10-05, 547 plans, 1.69 GB compressed). For each provider we keep the rows for payer P's commercial plans on the 16 codes of the 2026 report and map the file's plan name to one label; the crosswalk is manual and listed in the code, and maps every commercial plan of the cluster to that label because the provider file names the product line, not the plan. From the payer file we keep prices for the five providers' NPIs on the same codes, pooled under one NPI per provider, so the pairing is at the TIN level (provider E lists eight NPIs). We pair on the record key. A key has several payer values when several plans in the cluster price it; the primary measure pairs each provider line with the nearest payer value of the same kind, dollars with dollars and percent with percent, and we also report the pairing of each provider line with every payer value of its key. Both measures report exact match, within 10%, and more than 50% apart, over the pairs where both sides give the same kind of value.

Synthetic attribution. For each paired key with a provider dollar amount we sign one record with that amount as the agreed value, with prototype keys on both sides. We then build two copies of the signed value, one labeled provider and one labeled payer, alter 10% of the lines in each copy at random by $\pm 10\%$, $+50\%$, or -50%, verify both copies, and report the attribution of each difference. Both copies start from the record, so the run is a functional test of the attribution step and says nothing about real agreement; the payer never agreed the signed value.

Key rotation. On the running network we commit a record with a payer signing key, delegate a new key, revoke the old one, and repeat the commit with each key; we then attempt a backdated revocation, a root replacement before its delay, and a root replacement after a veto.

Discovery. We record how many provider files could be found and parsed. The CMS locator file (`cms-hpt.txt`) at

Table 1: Full-sheet run cost: one provider file, 20 payers, prototype keys on both sides. Rates and per-record figures are measured on the 187,025 records of the final run; figures for the sheet apply them to all 262,535 records.

Records signed (fixed / percent of charges)	262,535 (201,394 / 61,141)
Signing, both sides, final run	202 s (1,852 signatures/s)
Commit, 100 records per transaction	61 records/s (72.3 min for the sheet)
Gas per record	310,367
Transaction input per record	834 bytes (156 MB in the run)
Validator storage per record / for the sheet	17,945 bytes / 4,711 MB
Clean copy: match / amount differs / needs chargemaster	349,258 / 14,006 / 426
Altered copy: rows altered / caught / already differed / not comparable	3,584 / 3,454 / 128 / 2

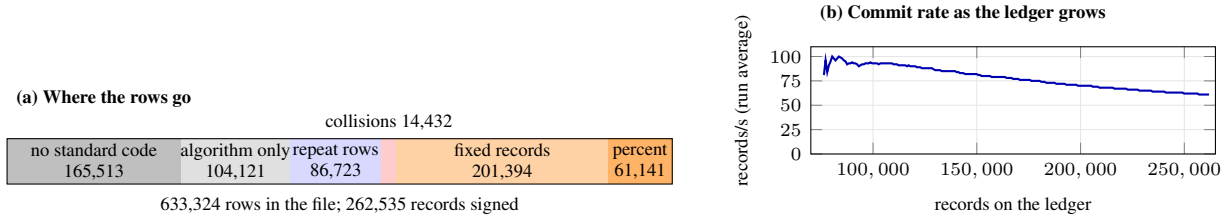


Figure 6: Full-sheet run. (a) Of 633,324 rows, those with a standard code and a value reduce to 262,535 signed records; repeat rows carry the key and value of an earlier row, collisions carry a different value for a key already signed. (b) The cumulative commit rate of the final run, which started with 75,510 records on the ledger and fell from 92 to 61 records per second.

each provider's domain names the file.

All code, the contract, the parsers, the run output, and the contract tests (sixteen, one per rule above) are in the repository [34].

Results

Full-sheet run

The file has 633,324 rows (Figure 6a). 165,513 rows carry only a chargemaster, revenue, or NDC code and no CPT, HCPCS, or MS-DRG code; a record needs a code both parties use, so these rows are outside the record format. The remaining 467,811 rows name 20 payers, and 104,121 of them give an algorithm and no dollar or percent value. The 363,690 rows with a value reduce to 262,535 records, 201,394 fixed and 61,141 percent of charges: 86,723 rows repeat the key and value of an earlier row, and 14,432 rows carry a different value for a key already signed, on 14,292 keys that hold 30,636 rows between them. These are items with the same billing code, setting, class, and unit but a different list price and a different fixed amount, which the key fields of the public formats do not separate; the record on such a key holds the first row's value.

Table 1 gives the cost. The run that produced it committed 187,025 records; 75,510 were on the ledger from two earlier runs that were interrupted, and each validator held 2.52 GB before it started. Signing ran at 1,852 signatures per second. Commits ran 100 records per transaction with eight transactions in flight, at 61 records per second over the run, and the rate fell as the ledger grew (Figure 6b): 92 records per second over the first 20,000 records of the run. The minutes for the sheet, 72.3, and its storage, 4,711 MB per validator, are the run's rate and per-record storage applied to all 262,535 records. Gas is a measure of execution work; the network charges no fee.

Verification of the clean file returned 349,258 matches, 14,006 amount-differs verdicts, and 426 needs-chargemaster verdicts; both kinds are the key collisions, the latter being collision rows with a dollar amount on a percent-of-charges key. In the altered copy, 3,584 rows had their signed value changed; 3,454 changed from match to amount differs, 128 were collision rows that differed before the change, and 2 were needs-chargemaster rows. No altered row that matched before the change matched after it. This run is self-consistency: the records were minted from the rows they are checked against, so a match is expected. It establishes that the record format covers every row with a standard code and a value, that the key holds for all but 14,432 rows, and the cost of signing a full sheet.

Table 2: Five providers against payer P's file on the 16 codes of the 2026 report, nearest payer value per provider line. Keys: distinct record keys on each side. Pairs: provider lines with a payer value for the same key; comparable: both sides give the same kind of value. Percentages are of comparable pairs.

Provider	File date	Codes	Provider keys	Payer keys	Pairs	Comparable	Exact %	Within 10%	Over 50%
A (Colorado)	2025-11-01	0	0	44	0	0	–	–	–
B (Colorado)	2025-11-01	16	19	73	25	19	57.9	94.7	0.0
C (Illinois)	2026-11-03	8	8	16	8	8	37.5	100.0	0.0
D (Oregon)	2026-03-23	16	46	24	16	5	0.0	100.0	0.0
E (California)	2026-04-01	8	8	18	3	1	100.0	100.0	0.0

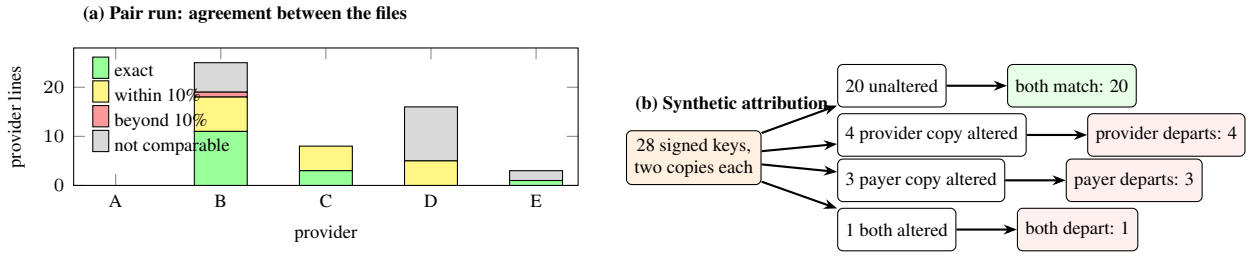


Figure 7: (a) Per provider, the comparable pairs by agreement and the pairs that cannot be compared; provider A has no pair. (b) The synthetic run: every altered copy is attributed to its side, and every unaltered pair matches the record on both sides.

Discovery

Of ten health systems tried on 2026-10-27, four served the CMS locator file. Of ten tried on 2026-11-03, four served a tall CSV, four served JSON, one a wide CSV, and one an empty archive; the JSON files are valid under the rule and our parser does not read them. On the payer side, one national payer publishes files of 9 GB and more per plan, which we did not download; payer P publishes 5,697 in-network files by plan cluster of about 1.7 GB each, with no state in the name. The pair run therefore covers one payer.

Pair run on real data

Table 2 gives, per provider, the keys on each side and the agreement on paired keys; Figure 7a shows the counts. Payer P's file lists 468 prices for the five providers' NPIs on the 16 codes, 232 after lines with the same key and value are merged, and a paired key has 1.53 payer values on average. Across providers, 52 provider lines paired; 33 carry the same kind of value on both sides, and 19 carry a dollar on one side and a percent of charges on the other and cannot be compared. Of the 33 comparable pairs, 15 (45.5%) matched exactly, 97.0% fell within 10%, and 0.0% differed by more than 50%. Paired against every payer value of the key instead of the nearest, there are 82 pairs, 50 comparable, of which 15 (30.0%) matched exactly, 74.0% fell within 10%, and 4.0% differed by more than 50%. Provider A has no pair: its rows for payer P on the 16 codes are all under a Medicare Advantage plan, which the Transparency in Coverage rule does not cover. By code group under the nearest-value measure, the inpatient codes give 17 comparable pairs with 29.4% exact, the gastroenterology codes 10 with 70.0%, and the orthopedic codes 6 with 50.0%; every group is 90% or more within 10%. The comparable sample is small, 33 pairs from four providers, of which provider B gives 19.

Synthetic attribution

28 paired keys had a provider dollar amount; one record was signed for each, and two copies were built (Figure 7b). The planned 10% alteration touched 5 provider copies and 4 payer copies under seed 7: 4 keys had only the provider copy changed, 3 only the payer copy, and 1 both. Verification attributed 4 differences to the provider copy, 3 to the payer copy, and 1 to both; the 20 unaltered keys matched the record on both sides. The construction guarantees this outcome when the clean copy equals the signed value; the run shows that the verifier delivers it.

Key rotation

Figure 8 gives the sequence on the running network. After the old key's revocation took effect, a commit with it reverted with `SignerNotAuthorized` and the same commit with the new key succeeded; `authorizedAt` returned true for the old key at the commit time of its earlier record and false after the revocation, so the earlier record

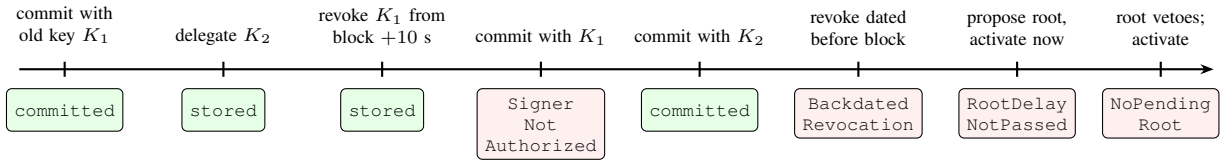


Figure 8: Key rotation on the running network, left to right. Shaded outcomes name the contract error that rejected the action; the record committed with K_1 before its revocation stays valid.

stays valid. A backdated revocation, a root activation before its delay, and a root activation after a veto each reverted with the error the figure names. Activation after the seven-day delay is covered by the unit tests, which move the clock, and was not run on the network.

Discussion

The record format holds every row of a real provider file that carries a standard code and a value, 363,690 of 633,324 rows. The key fields of the public formats identify a rate for all but 14,432 rows, on 14,292 keys; those are items that share a billing code, which the provider file knows by chargemaster code and the payer file does not know at all, and a comparison that pairs by code and payer alone meets them as several provider values for one payer value. Signing and committing a full sheet is a batch job of 72.3 minutes on one laptop, loading the registry into a verifier takes 572 s, and verifying the file against the loaded index takes 1.3 s.

The pair run against the 2026 report. On the 16 codes, five providers' files and payer P's file agree exactly on 45.5% of comparable pairs under the nearest-value pairing and 30.0% under the all-values pairing, against 9 to 12% in the 2026 report, and 97.0% and 74.0% within 10% against about half; Henderson and Mouslim's 77.4% exact agreement where both files had a price points the same way [14]. Three things separate our measure from the report's: we pair on the full key, so a provider line meets only payer values for the same plan label, setting, class, and unit, and the report's pairing is not published; we exclude the 19 of 52 pairs where one file gives a dollar and the other a percent of charges, which a ratio counts as disagreement; and our sample is 33 comparable pairs from four providers against one payer, so each percentage moves by three points per pair. The result does not contradict the report; it suggests that much of what the report measures is definitional, the plan, setting, class, unit, and date on which the two files differ, which are the fields the record fixes. What remains after pairing on the full key is what a signed record would attribute to one file.

Scale. The run gives 17,945 bytes per record on each validator and 61 records per second with the ledger between 75,510 and 262,535 records. About 6,000 hospitals, each with a sheet of this size, give 1.6 billion records, about 28 TB per validator, and about 301 days of commits at the measured rate, before any payer-side record and before the decline in Figure 6b continues. The prototype stores the whole record on chain; the bodies are already in the public files, so a deployment can store the record id and the two signatures only, which we have not built or measured.

Why a replicated ledger. Kuo and colleagues' conditions apply: providers, payers, and CMS each produce their own record of the same term; the history must be append-only, because a rate that changes mid-year must keep its earlier record; and no party is accepted by the others as the operator of the record. A CMS-run signed log with a hash chain would give the same inclusion guarantee. A lighter design exists too: each party signs one Merkle root per sheet per contract version, each published line carries an inclusion proof, and a one-operator log with witnesses detects equivocation; that is one pair of signatures per contract instead of one per rate, and no consensus layer. The per-record form buys two things over it: a mid-year change to one rate is one new record rather than a new sheet, and a verifier needs only the key, not the counterparty's sheet. Whether those are worth a replicated ledger is the governance choice, and whether the regulator should also operate the log is a policy question this paper does not settle. Table 3 gives the trust model as built. The registrar is a single point of trust for identity binding, limited by the delay and veto: it cannot sign a rate, remove a record, or re-key an entity without a week's notice. Three of four validators can reorder or rewrite history, and two can halt the chain; a deployment needs more validators, drawn from parties with opposed interests, or witness checkpoints of the chain head.

Privacy and market effects. Every validator reads every record in plaintext. The rates are public once published, but a record is committed before publication, so a validator run by a payer or provider association sees a competitor's amendments and their timing early; a deployment therefore sets read access as policy and can hold the record body off chain. A shared ledger of rates among competitors also raises the questions any exchange of price data among

Table 3: Trust model of the prototype: what each actor can do and what the registry prevents or detects.

Actor	Can do	Prevented or detected
Rate signer (either party)	sign any value for its side	cannot commit alone; record needs the counterparty's signature
Signing-key holder (vendor)	sign within the delegation window	revocation from block time; records before it stand
Thief of a root key	delegate new keys; veto root replacement	loss is recoverable, theft is not (no registrar override)
Relayer	submit, delay, or withhold a signed record	no authority; a withheld record is resubmitted by either party; a signed draft has no deadline (limitation)
Registrar	bind a root; propose a replacement	cannot sign a rate or remove a record; replacement waits seven days under a veto
One validator	see every record; propose blocks in turn	cannot forge a signature; timestamp bounded by the block period
Three of four validators	reorder or rewrite history	not prevented; needs more validators or external witnesses

competitors raises, which a deployment must answer before an association joins as a validator.

Adoption. The second signature does not exist today, and the party that must add it is the one whose file is less likely to be checked. The hospital rule could require that a reported rate cite a jointly signed record, as the CY2024 and CY2026 rules added and extended the affirmation statement [3, 4], and enforcement under 45 CFR 180.90 already covers hospital file content; the payer side needs the three departments that issue the Transparency in Coverage rule. A payer has its own reason to ask for the record as a contract term, as a defense against a provider file that overstates a rate, and a provider association could run the registry for its members and invite payers. Under any route the parties negotiate as now and add a signature step before publication. Where they do not agree a value, no record exists, both lines verify as no record, and the files carry the disagreement in the open. A record signed with a stolen key stands until both parties sign a superseding record.

Limitations. Both signatures in every run come from prototype keys, so no run shows that a file is correct, only that a file matches the record it was minted from; the synthetic run tests attribution on two copies of the signed value. The chargemaster version signed is the SHA-256 of the provider file itself, which carries the list prices; a deployment would sign the chargemaster as a separate file. The record carries a contract version and dates that no public file holds, so the runs set them to one label and one date. The record covers fixed, percent-of-charges, and percent-of-reference rates and not the other contract terms listed in the Background; algorithm-only rows (104,121 here) and leased networks, where a third party stands between the provider and the plan, are outside it. The hospital rule covers hospitals, and the Transparency in Coverage rule does not cover Medicare Advantage or Medicaid managed care, so for those rates only one side publishes. A rate signature carries no deadline, so an abandoned draft that both parties signed can be committed by anyone while both keys are valid. The pair run covers one payer, five providers, and 16 codes, after a manual plan crosswalk and TIN-level pooling, and the provider files are dated between 2025-11-01 and 2026-11-03 against a payer file dated 2026-10-05, so part of the disagreement can be a rate update. A national reproduction first needs a parser for the JSON provider format and a way to handle payer files of 9 GB and more. The registry cannot detect a rate that both parties signed wrongly.

Conclusion

On one real provider file the record format covers every row with a standard code and a value, the key holds for all but 14,432 rows, and signing the sheet is a batch job of 72.3 minutes on one laptop. On five providers and one payer the files agree exactly on 45.5% of comparable pairs under the nearest-value pairing and 30.0% under the all-values pairing, on the 16 codes of the 2026 report; pairing on the full record key appears to remove much of the disagreement the report measures, which a comparison of both pairings on the same files would have to confirm. With a jointly signed record behind each rate, a verifier attributes a difference to the file that departs from the record, as the synthetic run shows on two copies of the signed values; no comparison of the two files can do that today.

References

[1] Centers for Medicare & Medicaid Services. Medicare and Medicaid Programs: CY 2020 Hospital Outpatient PPS Policy Changes and Payment Rates and Ambulatory Surgical Center Payment System Policy Changes and Payment Rates. Price Transparency Requirements for Hospitals To Make Standard Charges Public. Final rule. Fed Regist. 2019 Nov 27;84(229):65524–65606. Codified at 45 CFR Part 180.

[2] Internal Revenue Service, Employee Benefits Security Administration, Centers for Medicare & Medicaid Services. Transparency in Coverage. Final rule. Fed Regist. 2020 Nov 12;85(219):72158–72310.

[3] Centers for Medicare & Medicaid Services. Medicare Program: Hospital Outpatient Prospective Payment and Ambulatory Surgical Center Payment Systems; Quality Reporting Programs; Hospital Price Transparency. Final rule with comment period. Fed Regist. 2023 Nov 22;88(224):81540. CMS-1786-FC.

[4] Centers for Medicare & Medicaid Services. Medicare Program: Hospital Outpatient Prospective Payment and Ambulatory Surgical Center Payment Systems (CY 2026). Final rule. Fed Regist. 2025 Nov 25;90:53448. CMS-1834-FC. Effective 2026 Jan 1.

[5] Centers for Medicare & Medicaid Services. Gag Clause Prohibition Compliance Attestation [Internet]. Baltimore (MD): CMS; 2024 Sep 19 [cited 2026 Nov 6]. Available from: https://www.cms.gov/marketplace/about/oversight/other-insurance-protections/gag-clause-prohibition-compliance-attestation

[6] Patton K, Ikard M, Browne J, Oakes A. The majority of hospital and health plan reported rates diverge by more than 10% [Internet]. Trilliant Health; 2026 Apr 30 [cited 2026 Nov 3]. Available from: https://www.trillianthealth.com/market-research/studies/the-majority-of-hospital-and-health-plan-reported-rates-diverge-by-more-than-10

[7] Centers for Medicare & Medicaid Services. Hospital price transparency CSV data dictionary, version 3.0 [Internet]. [cited 2026 Nov 3]. Available from: https://github.com/CMSgov/hospital-price-transparency/tree/master/documentation/CSV

[8] Centers for Medicare & Medicaid Services. Transparency in coverage: in-network rate file schema [Internet]. [cited 2026 Nov 3]. Available from: https://github.com/CMSgov/price-transparency-guide/tree/master/schemas/in-network-rates

[9] Sinaiko AD, Rosenthal MB. Increased price transparency in health care: challenges and potential effects. N Engl J Med. 2011;364(10):891–894. doi:10.1056/NEJMp1100041

[10] Mehrotra A, Chernew ME, Sinaiko AD. Promise and reality of price transparency. N Engl J Med. 2018;378(14):1348–1354. doi:10.1056/NEJMhpr1715229

[11] Gondi S, Beckman AL, Ofoje AA, Hinkes P, McWilliams JM. Early hospital compliance with federal requirements for price transparency. JAMA Intern Med. 2021;181(10):1396–1397. doi:10.1001/jamainternmed.2021.2531

[12] Jiang JX, Polsky D, Littlejohn J, Wang Y, Zare H, Bai G. Factors associated with compliance to the hospital price transparency final rule: a national landscape study. J Gen Intern Med. 2022;37(14):3577–3584. doi:10.1007/s11606-021-07237-y

[13] Wang Y, Meiselbach M, Anderson GF, Bai G. Hospital pricing information consistent between Transparency-in-Coverage data and other commercial data sources. Health Aff Forefront. 2023 Nov 8. doi:10.1377/forefront.20231108.269718

[14] Henderson MA, Mouslim MC. Cross-validation of insurer and hospital price transparency data. Am J Manag Care. 2024;30(8):e247–e250. doi:10.37765/ajmc.2024.89594

[15] Turquoise Health. 2025 price transparency impact report [Internet]. 2025 Oct 29 [cited 2026 Nov 3]. Available from: https://turquoise.health/resources/reports/2025-price-transparency-impact-report

[16] PatientRightsAdvocate.org. Eighth hospital price transparency compliance report [Internet]. 2026 Sep [cited 2026 Nov 3]. Available from: https://www.patientrightsadvocate.org/eighth-annual-hospital-price-transparency-compliance-report-september-2026

[17] Consolidated Appropriations Act, 2021. Pub L No. 116-260, Division BB, Title I, Sec. 111 (advanced explanation of benefits) and Sec. 112 (good faith estimate). 2020 Dec 27.

[18] HL7 International, Financial Management Work Group. Da Vinci Patient Cost Transparency Implementation Guide, version 2.0.1 (STU 2) [Internet]. Ann Arbor (MI): HL7 International [cited 2026 Nov 6]. Available from: https://hl7.org/fhir/us/davinci-pct/

[19] Kuo TT, Kim HE, Ohno-Machado L. Blockchain distributed ledger technologies for biomedical and health care applications. J Am Med Inform Assoc. 2017;24(6):1211–1220. doi:10.1093/jamia/ocx068

[20] Change Healthcare. Change Healthcare announces general availability of first enterprise-scale blockchain solution for healthcare [press release on the Internet]. PR Newswire; 2018 Jan 8 [cited 2026 Nov 3]. Available from: https://www.prnewswire.com/news-releases/change-healthcare-announces-general-availability-of-first-enterprise-scale-blockchain-solution-for-healthcare-300578989.html

[21] Kalina IM. Another big blockchain initiative announced in health care insurance industry [Internet]. National Law Review; 2019 Feb 6 [cited 2026 Nov 3]. Available from: https://natlawreview.com/article/another-big-blockchain-initiative-announced-health-care-insurance-industry

[22] Jasper KA. 5 healthcare groups sign up for blockchain credentialing [Internet]. Ledger Insights; 2018 Nov 9 [cited 2026 Nov 3]. Available from: https://www.ledgerinsights.com/healthcare-blockchain-credentialing-provider/

[23] Zhou L, Wang L, Sun Y. MIStore: a blockchain-based medical insurance storage system. J Med Syst. 2018;42(8):149. doi:10.1007/s10916-018-0996-4

[24] Laurie B. Certificate transparency. Commun ACM. 2014;57(10):40–46. doi:10.1145/2659897

[25] Laurie B, Langley A, Kasper E. Certificate Transparency. RFC 6962. Internet Engineering Task Force; 2013 Jun. doi:10.17487/RFC6962

[26] Google. Trillian: a transparent, highly scalable and cryptographically verifiable data store [software on the Internet]. [cited 2026 Nov 6]. Available from: https://github.com/google/trillian

[27] Birkholz H, Delignat-Lavaud A, Fournet C, Deshpande Y, Lasker S. An architecture for trustworthy and transparent digital supply chains. RFC 9943. IETF; 2026 Jun.

[28] Haeberlen A, Kouznetsov P, Druschel P. PeerReview: practical accountability for distributed systems. In: Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP); 2007 Oct; Stevenson, WA. p. 175–188. doi:10.1145/1294261.1294279

[29] Li J, Krohn M, Mazières D, Shasha D. Secure untrusted data repository (SUNDR). In: Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI); 2004 Dec; San Francisco. p. 121–136.

[30] Zieglmeier V. Appending data to blockchain is not sufficient for non-repudiation of receipt. arXiv:2308.04781 [cs.CR]. 2023 Aug 9.

[31] Bloemen R, Logvinov L, Evans J. EIP-712: Typed structured data hashing and signing [Internet]. Ethereum Improvement Proposals; 2017 Sep 12 [cited 2026 Nov 3]. Available from: https://eips.ethereum.org/EIPS/eip-712

[32] Department of Health and Human Services. Administrative Simplification: Rescinding the Adoption of the Standard Unique Health Plan Identifier and Other Entity Identifier. Final rule. Fed Regist. 2019 Oct 28;84:57621. CMS-0054-F.

[33] Hyperledger Besu. Configure IBFT 2.0 consensus [Internet]. [cited 2026 Nov 3]. Available from: https://besu.hyperledger.org/private-networks/how-to/configure/consensus/ibft

[34] Srivastava A. CORAL: Committed Ratesheet on Ledger [source code on the Internet]. 2026 [cited 2026 Nov 3]. Available from: https://github.com/AbhinavMir/coral

Working notes (remove before submission)

Everything built for this paper is recorded here. Dates are absolute. Numbers in the body come from `numbers.tex`, written by `scripts/make_numbers.py` from `results/*.json`. Repository: `https://github.com/AbhinavMir/coral`. Say provider, not hospital, except in a regulation name or a cited study. The body names no provider or payer (Abhinav, 2026-11-06): provider A = UCHealth Yampa Valley Medical Center (CO), B = UCHealth Memorial Hospital Central (CO), C = Advocate Christ Medical Center (IL), D = Oregon Health & Science University (OR), E = Alta Bates Summit Medical Center, Summit campus (CA); payer P = Aetna Life Insurance Company, plan cluster pl-1s7; the other national payer in Discovery is UnitedHealthcare; the 2026 industry study is Trilliant Health.

State (2026-11-06)

- Review round on 2026-11-06: five simulated reviewers (editor, methodology, domain, systems, devil's advocate) on the 2026-11-03 body. Unanimous: major revision. Fixed in this version: the synthetic "before signing" sentence was wrong (the payer copy is a clone of the provider line; `evaluate_pairs.py` line `q = replace(ln, source="payer")`), so the run is now described as two copies of the signed value and the attribution claim is restricted; Table 1 states that the rates come from the final run of 187,025 records and that sheet figures are per-record figures applied to 262,535; the all-values pairing (30.0% exact, 74.0% within 10%, 4.0% over 50%) is reported next to the nearest-value pairing; keys with several values (14,292) and repeat rows (86,723) are counted (`scripts/sheet_stats.py`, `results/sheet_stats.json`); the hand-typed "36%" became `\sheetPctShare` (37.7% of rows with a value); TIN-level NPI pooling, single-host Docker setup, Besu version, and block period are stated; index load time sits next to verify time; the Discussion reconciles the 2026 report with the paper's own 97% within 10% and with Henderson and Mouslim; a scale projection (6,000 hospitals: 1.6 billion records, 28 TB per validator, 301 days at 61 records/s) is given; the ledger is compared with a Merkle-root-per-sheet signed log; a trust table (Table 3) lists each actor; root theft, the missing signature deadline, and past-dated supersession are stated; regulatory facts corrected (CY2026 rule, gag clause attestation, no file is sent to CMS, three departments enforce Transparency in Coverage, HPID rescinded, Medicare Advantage and Medicaid managed care outside Transparency in Coverage); contract forms the record does not cover are listed; company names removed from Related work; references in order of first citation, 34 entries, Certificate Transparency split into the ACM article and RFC 6962, Trillian cited; 12 references added and verified against primary pages (CY2026 rule 90 FR 53448, gag clause page, HPID rescission 84 FR 57621, Sinaiko 2011, Mehrotra 2018, Gondi 2021, Pub L 116-260 secs 111 and 112, Da Vinci PCT 2.0.1, MIStore 2018, RFC 6962, Trillian).
- Eight figures added (all TikZ or pgfplots inside `abstract.tex`; chart data in `results/commit_rate.dat` and `results/pairs_bars.dat`, written by `make_numbers.py`): the two-file problem against CORAL; record field mapping; key hierarchy and time rules; commit path and supersession; verdict tree and pair matrix; row distribution and commit-rate decline; pair-run bars and synthetic attribution flow; key-rotation timeline. Table 3 (code group) became one sentence. Body with references is 10 pages, the AMIA limit; references set in `\footnotesize`, paragraph skip 0.4em.
- Open for Abhinav: the title (the systems reviewer asks for "permissioned" rather than "private" and the editor asks to lead with the jointly signed record); whether to keep the ledger framing or present the signed-log design as the primary one; whether the abstract should carry result figures (the editor asks for them; Abhinav's rule is no raw counts in the abstract). Not fixed in code: a deadline field in the rate signature, a registrar override for root theft, off-chain record bodies, the 7-day activation on the network (unit tests only). Reviewer reports are in the session transcript, not in the repository.

State (2026-11-03)

- Paper body written: Introduction, Background, Related work, Methods, Results, Discussion, Conclusion, 23 references. Title unchanged; the review's suggestion (do not lead with "Private Blockchain") is not applied. Abhinav decides.
- Contract rewritten on 2026-11-03 after the review: revocation bound to block time, root replacement with a seven-day delay and a root veto, supersedes field, overlapping windows rejected, batch commit, batch read, drug unit and billing class in the key. 16 tests pass.

- Runs on 2026-11-03: full sheet (`results/evaluate_sheet.json`), five providers against one Aetna file on 16 codes plus the synthetic attribution run (`results/evaluate_pairs.json`), key lifecycle on the network (`results/key_events.json`). All numbers in the body come from these files.
- Not done: UnitedHealthcare payer side (files are 9 GB and more per plan); JSON provider files (four of ten systems); an on-ledger proposal record; witness checkpoints; a separate chargemaster file.

Problem

- Providers publish negotiated rates under 45 CFR 180. Payers publish in-network rates under Transparency in Coverage (85 FR 72158). Each file is produced by one party. No file carries the other party's signature.
- Trilliant Health, 30 April 2026: 450 hospitals, Aetna and UnitedHealthcare, 16 codes (8 MS-DRG: 189, 190, 193, 280, 291, 392, 552, 641; 5 GI CPT: 43239, 45380, 45385, 47562, 49650; 3 orthopedic CPT: 29827, 29881, 29916), data processed August 2025. Exact 9.0% inpatient, 10.8% GI, 11.7% orthopedic. Within 10%: 50.8%, 41.7%, 47.4%. Over 50% apart: 17.9% inpatient and GI. Ratio = hospital rate / plan rate; exact 1.0, near 0.9–1.1, material under 0.5 or over 1.5. The code list is in an image on the report page, not in its text.
- UCHealth Yampa Valley Medical Center file (2025-11-01, CMS 3.0.0, NPI 1790787307): 633,324 rows; every row billing class facility; settings inpatient 229,285 and outpatient 404,039; methodologies: percent of total billed charges 228,164, other (algorithm) 178,531, fee schedule 168,658, case rate 57,918, per diem 53; 30 payer/plan pairs; 165,513 rows have only CDM, RC, or NDC codes.

Terms

- Negotiated rate: the price a provider and a payer agreed for one code. Contractual. The CMS file calls it the payer-specific negotiated charge.
- Chargemaster: the provider's list price for every item. Set by the provider alone. The CMS file calls it the gross charge. A percent-of-charges rate is a percent of this number.
- Setting: inpatient or outpatient (CMS column; "both" expands to one line per setting). Billing class: facility or professional (CMS column; TiC `billing_class`). Drug unit: CMS `drug_unit_of_measurement` and `drug_type_of_measurement`, joined as "1 ML". TiC has no drug unit.
- Record key: payer, plan, provider TIN, NPI, code, modifier, setting, billing class, unit. Not in either file and not in the record: place of service beyond the setting split, revenue code, bundling, multiple-procedure reductions, outliers, carve-outs, per-diem tiers, escalators.
- A percent-of-charges row in the CMS file carries both the percent and the dollar result. The contract term is the percent; the record is percent of charges and the verifier recomputes the dollar from the row's gross charge. Signing the dollar instead produced 81,470 false key collisions on the first trial (2026-11-03); signing the percent leaves 14,432.

Prior art (searches 27 October and 3 November 2026)

- No paper or product found on jointly signed negotiated rates or a ledger that holds rate records.
- Claim and directory networks: Change Healthcare on Hyperledger Fabric (2018), Synaptic Health Alliance (provider directory), ProCredEx (credentials), MediLedger 340B (2021). None holds a rate with both signatures.
- File comparisons: Jiang et al. JGIM 2022 (compliance); Wang et al. Health Affairs Forefront 2023 (TiC consistent with commercial sources); Henderson and Mouslim AJMC 2024 (insurer vs hospital cross-validation); Turquoise 2025 impact report; PatientRightsAdvocate eighth report (September 2026, 49.4% of 2,000 hospitals compliant). No Jiang/Bai hospital-versus-insurer paper exists; the review's pointer was wrong.
- Transparency logs: Certificate Transparency, Trillian, SCITT (now RFC 9943, June 2026). PeerReview, SUNDR. Zieglmeier arXiv 2308.04781: an append is not proof of receipt.

- CY2024 OPPS rule (88 FR 81540): affirmation statement and estimated allowed amount; CY2026 replaced the estimated allowed amount with median, 10th, and 90th percentile allowed amounts from 1 January 2026.

Keys

- Identity roots: provider TIN (the Aetna file carries the EIN per provider group, so the payer side can name the same entity); payer id (prototype: opaque string; deployment: HIOS issuer id or EIN; the HPID was rescinded in 2019 and X12 does not assign payer ids). Registrar: CMS, binds entity id to root key once, never signs a rate.
- Root key offline, signs delegations, revocations, and root vetoes only. Signing keys delegated for a window. `authorizedAt(entity, key, t)` answers from the ledger.
- Validity judged at the block timestamp of the commit. `revoke` rejects `effectiveFrom` before `block.timestamp` (`BackdatedRevocation`), so a root cannot repudiate a record its key signed. The app's rotate action revokes from now plus ten seconds.
- Root replacement: `proposeRoot` (registrar), `activateRoot` after `ROOT_DELAY` = 7 days, `vetoRoot` by the current root (EIP-712 `RootVeto`). Old delegations stay until the new root revokes them.
- EIP-712 typed data for every record; domain bound to chain id and contract address. Both parties sign the same struct hash; the hash is the record id. First signature held off-ledger in `build/pending.json` until the second party countersigns.

Design

- Rate struct: 16 fields (9 key, 7 value including `supersedes`). `rateId` hashes two `abi.encode` halves joined with `bytes.concat`, which equals one encode of all fields; the contract compiles with `viaIR` to avoid stack-too-deep.
- One live record per key: `keyRecords[keyHash]` lists ids; `_checkOverlap` rejects a window that overlaps a live window (`OverlappingWindow`); `supersedes` names the record to end, which must share the key and not be superseded already; the old record's effective end becomes the new start (`RateSuperseded` event; `effectiveEnd(id)`).
- Identifiers are `bytes32`. A string over 32 bytes (the Yampa file has the plan "Narrow Network Exclusive Plan (EPO)", 35 bytes) is stored as its keccak hash; the ledger cannot return the text. `keyform` puts both sides of a comparison in the ledger's form (text, or hex of the hash). Found on 2026-11-03 when the resumed sheet run hit `AlreadyCommitted` on records the index had read back under a different key.
- The full-sheet run was interrupted twice (a transient "latest block not found" from Besu at 75,200 records; then the key-form defect) and resumed; the script skips keys already on the ledger, and `retry` covers null RPC answers during block import. Commit rate and per-record figures come from the third run (187,025 records); the minutes-for-the-sheet figure is records divided by that rate. The rate fell with ledger size: the first run on an empty ledger ran between 165 and 190 records per second; the third run ran 93 over its first 20,000 records and 61 over all of it. The verify step was rerun alone (`-skip-commit`) after the tamper step was corrected to alter the field the key's record holds (the percent when the record is percent of charges, else the dollar). The first version altered the dollar on every row, which a percent record does not sign, and reported 1,321 of 3,584 altered rows as missed; a second version altered the percent whenever the row had one and missed 5 rows; those were drug amounts of 3 or 4 cents that a 10 percent change rounded back to the same cent, so the tamper step now moves a value by at least one cent or one basis point. `make_numbers.py` asserts that no altered row still matches.
- `commitRates` commits a batch atomically. Besu's default transaction size limit (128 KB) allows about 100 records per transaction at 834 bytes each; 250 failed with "Transaction size exceeds the maximum allowed size".
- `getRates(ids[])` returns bodies and commit data for many ids; the client reads 500 per call. Without it, loading 262k records took two RPC calls each.

- Verifier verdicts: match, amount differs, no record, expired, ambiguous (cannot occur now), needs chargemaster. Percent lines compared on the percent when the file gives it.

Build

- `contracts/RateRegistry.sol` (Solidity 0.8.24, viaIR, 11,901 bytes, 59 ABI entries). `coral/typed.py` (types, `Rate.key()`, `sign_root_veto`). `coral/chain.py` (`get_rates`, `commit_rates`, `propose/activate/v` `RateRecord.effective_end`, `covers`). `coral/verify.py` (`Line.key()`). `coral/mrf.py` (`Skipped` counts; `normalize_modifiers`; unit and billing class; payer parser: gzip, code filter, remote `location` references counted as `provider_reference_remote`, no service code counted as `setting_unknown`, place of service 21 inpatient and 19/22/23/24 outpatient, per diem accepted as an amount). `coral/match.py` keys on `Line.key()`.
- Network: Hyperledger Besu 26.10.2, IBFT 2.0, four validators in Docker, chain id 2741, zero base fee. `Chain.logs` scans from the deploy block in 4000-block ranges (Besu caps `eth_getLogs` at 5000 blocks).
- Scripts: `bootstrap.py` (deploy, register three entities), `seed_examples.py` (ten examples plus the chargemaster hash; J1650 carries unit "1 ML"), `evaluate_sheet.py` (full sheet; registers every payer in the file; signs with a process pool; batches of 100, eight in flight; verifies clean and 1% altered copies; disk delta of the CMS validator), `evaluate_pairs.py` (five providers vs Aetna on 16 codes; synthetic attribution), `make_numbers.py`. `evaluate_provider.py` (the 200-record run of 2026-10-27) was removed; its output stays in `results/evaluate_provider.json`.
- App: provider/payer page (Negotiated rates with one search box, Pending, Propose with billing class, unit, and "replaces record", Chargemaster, Keys) and CMS page (Registry, Verify a published file with unit and class columns and skipped counts). The verify endpoint loads the ledger once into a `RateIndex`.
- Tests (16): commit and read, duplicate, unauthorized key, wrong-side key, revocation, backdated revocation rejected, delegation window, root cannot sign, domain binding, root replacement waits and can be vetoed, root replacement after delay, overlapping window rejected, supersede amends mid-window, batch commit (one bad signature fails the batch), verify fixed, verify percent of charges.

Data

- Provider files (`data/raw/`, not committed): UCHealth Yampa Valley (141 MB); `others/`: OHSU (284 MB, 2026-03-23, NPI 1609824010), Advocate Christ (143 MB, 2026-11-03, NPIs 1548375082, 1386188068, 1922542604), UCHealth Memorial Hospital Central (267 MB, 2025-11-01, NPI 1144397134), Alta Bates Summit, Summit campus (15 MB, 2026-04-01, eight NPIs). All CMS 3.0.0 tall CSV.
- Discovery, 2026-11-03: tall CSV from OHSU, Advocate, UCHealth, Sutter; JSON from NYP, Houston Methodist, Stanford, BJC; wide CSV from UW Health; empty zip from Intermountain. 2026-10-27: four of ten served `cms-hpt.txt` (NYP, UCHealth, OHSU, Houston Methodist).
- Payer files (`data/raw/tic/`): UHC index `https://transparency-in-coverage.uhc.com/api/v1/uhc/blob` (83,731 blobs, 2,977 in-network; commercial files 9.0–9.6 GB each; not downloaded). Aetna index via `health1.aetna.com/h` to `latest_metadata.json` (5,697 in-network files, 2026-10-05, by plan cluster); downloaded `pl-1s7-tr25` (547 plans, 414 Aetna Choice POS II, 1.69 GB gz). The file has inline `provider_references` with NPIs and EIN TINs.
- Aetna commercial rows per provider file: Yampa (Aetna, Commercial); Memorial (Aetna Core/Meritain, Commercial; Aetna ASA, Commercial not used); Advocate (Aetna, Commercial); OHSU (AETNA HEALTHCARE, AETNA); Alta Bates (Aetna, HMO/PPO). Crosswalk label on both sides: "Aetna commercial".

Results are in the body (Tables 1–3) and in `results/`. Earlier run, 2026-10-27, superseded: 200 UHC Broad Networks records one per block, 258,923 gas each, 1.99 s each, 200/200 match, 20/20 tampered caught.

- Pair run: the Aetna file gives a percent of charges for many of the 16 codes where the provider file gives a dollar (OHSU 11 of 16 pairs; Memorial 6 of 25). These pairs are reported as not comparable; the percentages are of comparable pairs only. Each provider line is paired with the nearest payer value of the same kind because the crosswalk collapses every Aetna commercial plan to one label (1.53 payer values per paired key); the line-by-line measure against every payer value is reported beside it. Yampa Valley's Aetna commercial rows have none of the 16 codes (its Medicare Advantage rows do), so it contributes no pairs.
- Synthetic attribution: 28 paired keys had a provider dollar amount; 28 records signed; 4 provider-altered, 3 payer-altered, 1 both; attribution 20 both match, 3 payer departs, 4 provider departs, 1 both depart.
- Key events (`key_events.json`): backdated revoke reverted `BackdatedRevocation` (0x1532513f); commit with the revoked key reverted `SignerNotAuthorized` (0x3b94fe24); commit with the new key succeeded; `authorizedAt` true at the earlier commit time, false after; activation before the delay reverted `RootDelayNotPassed` (0xe90309b1); after the veto `NoPendingRoot` (0x2a01895e); root unchanged. The script rotates the PAYER:AETNA signing key in `build/keys.json`.

Review (2026-11-02, simulated PI) and what was done

- Abstract overclaims: rewritten to the runs. Done.
- Circular 200-record evaluation: replaced by the full-sheet run reported as self-consistency plus cost, with the key-collision and no-code findings. Done.
- `revoke` backdating: bound to block time. Done. `replaceRoot` without delay: delay and veto. Done. No correction path: supersedes and overlap rejection. Done.
- Drug unit and billing class in the key: done. Payer parser defects (remote references, missing service code): counted, not guessed. Done.
- Invented fields (contract version, start, end): stated in Methods and Limitations. Chargemaster is the MRF's own hash: stated in Limitations.
- Attribution as a synthetic run: done (`evaluate_pairs.py`, part 2).
- 450-hospital reproduction replaced by five providers, one payer, 16 codes, with discovery reported. Done. UHC side not done (file size).
- Related work added (Jiang 2022; Wang 2023; Henderson 2024; Turquoise; PRA; OPPS CY2024; Kuo 2017; ProCredEx; Zieglmeier). The Jiang/Bai comparison paper the review named does not exist.
- Writing: gas kept in Table 1 as a work measure with a note that the network charges no fee; EIP-712 named once in Methods; adoption paragraph added; "departs from the signed record" used. Title not changed.

Open items

- UHC payer side: one 9 GB file for the UHC plans in the Yampa file.
- JSON provider files: a parser for the CMS JSON schema would double the discovery rate.
- Item-level key: the 14,432 collision rows need a chargemaster item id in the key or a rule that one code has one rate.
- On-ledger proposal record (who refused to sign); witness checkpoints; a separate chargemaster file instead of the MRF hash.
- Cut to 10 pages and run a claim audit before submission.

Venue

- AMIA Annual Symposium 2027, Paper - Regular: 10 pages including references and figures, abstract 125–150 words, one column, Letter, Vancouver references. Symposium 6–10 November 2027, San Diego. Call for papers expected March 2027.
- Alternatives: IEEE ICBC 2027 (deadline 14 January 2027), Blockchain in Healthcare Today, Journal of Medical Systems.